

LING 425: Advanced Semantics Notes

Relations

Two-place relations

- Love
- Speak (between people and languages)
- Father of
- Be taller than
- Be the same age as
- $\geq$

Relations can be formally modeled as sets of ordered pairs:

- If A and B are sets, then R is a relation from A to B , if $R \subseteq A \times B$
- A is called the domain of R and B is the co-domain of R , i.e. domain to co-domain relation

Example:

Let $A = \{\text{Dorothy, Laura, Vi}\}$

Let $B = \{\text{French, Cantonese, Dutch, Swahili}\}$

Suppose that

- Dorothy speaks Cantonese and Swahili
- Laura speaks French, Cantonese, and Dutch
- Vi speaks Cantonese

Then, $R_{\text{speaks}} = \{\langle \text{Dorothy, Cantonese} \rangle, \langle \text{Dorothy, Swahili} \rangle, \langle \text{Laura, French} \rangle, \langle \text{Laura, Cantonese} \rangle, \langle \text{Laura, Dutch} \rangle, \langle \text{Vi, Cantonese} \rangle\}$

R is **reflexive** iff for all $x \in A : \langle x, x \rangle \in R$

R is **symmetric** iff for all $x, y \in A : \langle x, y \rangle \in R$ then $\langle y, x \rangle \in R$

R is **transitive** iff for all $x, y, z \in A : \text{if } \langle x, y \rangle \in R \text{ and } \langle y, z \rangle \in R, \text{ then } \langle x, z \rangle \in R$

Functions

A function is a type of (two-place) relation from A to B such that each member of A is paired with exactly one member of B

- I.e. relation with specific constraints
- Every member in domain is mapped to exactly one member in the codomain
 - Note that not everything in the codomain needs to have something mapped to it
 - A member in the domain cannot have multiple mappings to the codomain
 - Every member in the domain must be mapped to a member in the codomain
 - Note partial functions though
- Essentially a one-to-one relation/mapping

f is a partial function from A to B if f is defined for some members of A but not for others. Otherwise it is a total function

- Example: "spouse" function is only defined for people who are married -> partial
- Example: "date of birth" function (from people to dates) is a total function -> everyone has exactly one birthday
- Note that if f is a partial function, then it is a total function from a proper subset of A

Notation

- $f(x) = y$
- x is the argument of the function (input)
- y is its value (output)
- Truth value -> interpretation function

Predicate logic

Propositional logic formulas (sentences) have no internal structure -> doesn't get us very far in modeling natural language

Predicate logic allows us to model sub-parts of sentences

Lexicon of L_0

- Individual constants: a , b , c , etc. (often individual people)
- Individual variables: x , y , z , etc. (possibly also x_3 , x' , etc.)

- In comparison to propositional logic which uses meta-variables, these are part of the language
- Predicate symbols (words that start with a lower-case):
 - unary: e.g. happy
 - binary: e.g. loves
 - ternary: e.g. gives
 - etc. $\rightarrow$ n-ary
- Function symbols
- Connectives: $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\iff$
- Quantifier symbols: $\forall$, $\exists$
- Brackets and parentheses

Preliminary syntactic rules

Atomic formulas

- If π is a unary predicate (i.e., predicate with arity 1) and α is a term (i.e., individual constant or variable), then $\pi(\alpha)$ is an atomic formula
- If π is a binary predicate (i.e., predicate with arity 2) and α_1 and α_2 are terms, then $\pi(\alpha_1, \alpha_2)$ is an atomic formula
- Generalizing, if π is a predicate with arity n , and $\alpha_1, \dots, \alpha_n$ are terms, then $\pi(\alpha_1, \dots, \alpha_n)$ is an atomic formula

Semantics

Formulas of L_0 are interpreted relative to a model

A model M consists of

- Domain D (set of individuals)
- Interpretation function I , which maps each individual constant to a member of D (function from set of individual constants to D)
- Formally, we write: $M = \langle D, I \rangle$
 - $M_1 = \langle D_1, I_1 \rangle$
 - $M_2 = \langle D_2, I_2 \rangle$

Example: Suppose L_0 has individual constants j , p , g , and r and $D = \{\text{John, Paul, George, Ringo}\}$

Then I might be defined as follows:

- $I(j) = \text{John}$
- $I(p) = \text{Paul}$
- $I(g) = \text{George}$
- $I(r) = \text{Ringo}$

Function notation: big function mapping all of them instead of individually

Predicates are interpreted as sets

- I maps every unary predicate onto a set of individuals (subset of D)
 - Example:
 - $I(\text{guitarist}) = \{\text{John}, \text{Paul}, \text{George}\}$
 - $I(\text{drummer}) = \{\text{Ringo}\}$
 - $I(\text{woman}) = \emptyset$
- I maps every binary predicate on a set of pairs of individuals (subset of $D \times D$)
 - Example: $I(\text{envies}) = \{\langle \text{Paul}, \text{John} \rangle, \langle \text{John}, \text{Paul} \rangle, \langle \text{George}, \text{Paul} \rangle, \langle \text{George}, \text{John} \rangle\}$
- Formally:
 - If π is a predicate with arity 1, then $I(\pi) \subseteq D$
 - If π is a predicate with arity 2, then $I(\pi) \subseteq D \times D$
 - If π is a predicate with arity 3, then $I(\pi) \subseteq D \times D \times D$
 - etc

The denotation function $[[\]]^M$ is defined compositionally relative to a model $M = \langle D, I \rangle$, and assigns a denotation to every expression of the language (i.e., not just to formulas but also to terms)

- If α is a non-logical constant (individual constant or predicate symbol), then $[[\alpha]]^M = I(\alpha)$
- If π is a unary predicate and α is a term, then $[[\pi(\alpha)]]^M = T$ iff $[[\alpha]]^M \in [[\pi]]^M$
- If π is a binary predicate and α_1 and α_2 are terms, then $[[\pi(\alpha_1, \alpha_2)]]^M = T$ iff $\langle [[\alpha_1]]^M, [[\alpha_2]]^M \rangle \in [[\pi]]^M$
- Generalizing, if π is a predicate of any arity n and $\alpha_1, \dots, \alpha_n$ are terms, then $[[\pi(\alpha_1, \dots, \alpha_n)]]^M = T$ iff $\langle [[\alpha_1]]^M, \dots, [[\alpha_n]]^M \rangle \in [[\pi]]^M$

Functions

Adding more items to model $M = \langle D, I \rangle$:

$D = \{\text{John}, \text{Paul}, \text{George}, \text{Ringo}, \text{Yoko}, \text{Linda}\}$

$I(o) = \text{Yoko}$

$I(l) = \text{Linda}$

And adding **function symbol** `spouseOf` , which maps individual to their spouse

- As convention, function symbols will always end in `Of`

$I(\text{spouseOf}) = \{ \langle \text{John}, \text{Yoko} \rangle, \langle \text{Yoko}, \text{John} \rangle, \langle \text{Paul}, \text{Linda} \rangle, \langle \text{Linda}, \text{Paul} \rangle \}$

- Note that `spouseOf` is a partial function

Functional terms

We can now form complex **functional terms**, e.g., `spouseOf(j)` , `spouseOf(l)`

- Can be arguments of predicates like any other terms, e.g., individual constants such as `woman(spouseOf(j))` , `envies(spouseOf(p), spouseOf(j))`

Syntactic rule: Given any function γ with arity n , then:

$$\gamma(\alpha_1, \dots, \alpha_n)$$

- is a term, where $\alpha_1, \dots, \alpha_n$ is a sequence of expressions that are themselves terms

Semantic rule: If γ is a unary function symbol and α is a term, then:

$$[[\gamma(\alpha)]]^M = [[\gamma]]^M([[\alpha]]^M)$$

Syntactic rule (Identity): If α and β are terms, then $\alpha = \beta$ is an atomic formula

Semantic rule (identity): If α and β are terms, then $[[\alpha = \beta]]^M = T$ if $[[\alpha]]^M = [[\beta]]^M$ and F otherwise

- Examples: `spouseOf(l) = p` ; `p = r`

Connectives: L_0 has same syntactic and semantic rules for connectives as L_{prop}

Quantifiers

Examples of well-formed formulas with quantification:

- $\forall x [\text{guitarist}(x) \rightarrow \neg \text{woman}(x)]$
- $\exists y. \text{envies}(p, \text{spouseOf}(y))$

Syntactic rule: Given any variable u , if ϕ is a formula then $[\forall u. \phi]$ is a formula and so is $[\exists u. \phi]$

- Notation variants: $\forall u[\phi]$, $\exists u[\phi]$
- ϕ is called the scope of the quantifier

Variables, assignments, semantics

Atomic formula: any variable is **free**

- E.g. $\text{guitarist}(x)$

The free variables in ϕ are also free in all connectives, i.e., negation, conjunction, disjunction, and so on

All of the free variables in ϕ are free in $[\forall u.\phi]$ and $[\exists u.\phi]$ except for u

- Every occurrence of u in ϕ is **bound** in the quantified formula

A formula containing no free variables = **closed formula**

- Also includes a formula that contains no variables at all

A formula containing at least one free variable = **open formula**

Variables do not have a fixed interpretation in a model, i.e., they do not get their semantic variable from interpretation function but rather from assignment g

- Variables are like pronouns
- Free variables get a temporary denotation in the same way that free pronouns get a temporary denotation in a particular context
- So assignments are like pointers (that can change)

Assignments

Formally, an assignment is a function from the set of variables to D in a model M .

Example: Let L_{pred} have just 3 variables: x, y, z

- Then $g1$ and $g2$ are two assignments

$$g1 = \begin{bmatrix} x \rightarrow \text{John} \\ y \rightarrow \text{Yoko} \\ z \rightarrow \text{Linda} \end{bmatrix}$$

$$g2 = \begin{bmatrix} x \rightarrow \text{Linda} \\ y \rightarrow \text{Yoko} \\ z \rightarrow \text{Linda} \end{bmatrix}$$

Semantic values

Semantic values now always going to be relative to both model (including interpretation function and domain) and an assignment

Notation: $[[\alpha]]^{M,g}$ = the semantic value of α relative to M and g

- Variables: Semantic value determined by assignment g
- Constants: Semantic value still determined by interpretation function (as before)

Semantic rule:

- If α is an individual constant, then $[[\alpha]]^{M,g} = I(\alpha)$
- If α is a variable, then $[[\alpha]]^{M,g} = g(\alpha)$

Quantifiers are operators that **change assignment functions**

- A quantifier will only change the semantic value of its own (bounded) variable and leave all other (free) variables unchanged
- Notation: $g[u \rightarrow k]$
- Suppose u is a variable and k is an individual constant, i.e., $k \in D$
- Then we define $g[u \rightarrow k]$ as that assignment function g' that is exactly like g except that it maps u onto k

Example: Let g_1 be the same as defined above. What is:

1. $g_1[x \rightarrow \text{Paul}]$ - Assignment function g_1 is the same, but x maps to Paul
2. $g_1[y \rightarrow \text{Ringo}]$ - y maps to Ringo, everything else in g_1 stays the same
3. $(g_1[y \rightarrow \text{Ringo}])[y \rightarrow \text{Paul}]$ - y maps to Paul, everything else in g_1 stays the same (essentially take b as first step)

Semantic rules for quantifiers

Existential semantic rule: $[[\exists x.\phi]]^{M,g} = T$ iff there is an individual $k \in D$ such that:

$$[[\phi]]^{M,g[x \mapsto k]} = T$$

Universal semantic rule: $[[\forall v.\phi]]^{M,g} = T$ iff for all individuals $k \in D$ such that:

$$[[\phi]]^{M,g[v \mapsto k]} = T$$

Type theory

Compositionality problems

Mismatches between predicate logic and natural language

- "John envies Paul" $\rightarrow \text{envies}(j, p)$
 - Nothing in formula corresponds to the verb phrase (VP) 'envies Paul', i.e., as a constituent
- "Every dog barks" $\rightarrow \forall x[\text{dog}(x) \rightarrow \text{barks}(x)]$
 - Nothing in formula corresponds to determiner phrase (DP) 'every dog'

Problems for compositionality

- Want to take syntactic structures and interpret them
- Applicable to other constituents (terminal nodes in tree)
 - Presupposes that meaning of sentence follows its structure

Recursive definition of types

Basic types:

- e : entities
- t : truth values

Complex types - functional types:

- If σ and τ are types, then $\langle \sigma, \tau \rangle$ is also a type
- $\langle \sigma, \tau \rangle$ is the type of functions from type σ to type τ
 - Element σ has a type
 - Element τ has a type
- Examples
 - $\langle e, t \rangle$
 - $\langle e, e \rangle$
 - $\langle t, t \rangle$
 - $\langle e, \langle e, t \rangle \rangle$
 - $\langle \langle e, t \rangle, t \rangle$
 - $\langle \langle e, t \rangle, \langle e, t \rangle \rangle$

Type $\langle e, t \rangle$

This is the type of functions from entities to truth values

- A function of type $\langle e, t \rangle$ is the characteristic function of a set of entities
- Such a function returns T if the input entity is a member of the set denoted by the predicate, F otherwise

Example

- Let $D = \{\text{John, Paul, George, Ringo}\}$
- $\text{guitarist}^{M,g} = \{\text{John, Paul, George}\}$
- The characteristic function of the set of guitarists is:
 - $f_{\text{guitarist}} = \text{big set}$
 - $\text{John} \rightarrow \text{T}$
 - $\text{Paul} \rightarrow \text{T}$
 - $\text{George} \rightarrow \text{T}$
 - $\text{Ringo} \rightarrow \text{F}$

So, type $\langle e, t \rangle$ is the type of characteristic functions of sets, i.e., one-place (unary) predicates

- Examples from natural language
 - Intransitive verbs: sings, walks
 - Non-relational nouns: guitarist, drummer
 - Most adjectives: happy, blue

Type $\langle e, e \rangle$

Type of functions from entities to entities

Examples

- bestFriendOf (function from individuals to individuals)
- Plurals

Type $\langle t, t \rangle$

Type of functions from truth values to truth values

Examples

- Negation

Type $\langle e, \langle e, t \rangle \rangle$

Type for representing two-place predicates (binary relations)

- I.e., functions from entities to functions from entities to truth values
- Essentially takes two entity arguments but one at a time and returns a truth value

Examples

- Transitive verbs: envies
- Relational nouns: sister

Currying

Currying is a process of transforming a single function taking multiple inputs (arguments) into multiple functions each taking one argument

Example

- Let $D = \{\text{John, Paul, George, Ringo}\}$
- $\text{admires}^{M,g} = \{\langle \text{John, Ringo} \rangle, \langle \text{Paul, John} \rangle, \langle \text{George, John} \rangle, \langle \text{George, Paul} \rangle\}$
- Currying from left to right
- Currying from right to left
 - Takes second element of ordered pair as first argument and first element of ordered pair as second argument

Note that English, transitive verbs are curried from right to left

- If person admires John $\rightarrow$ function rather than John admires person
 - John is admired by Paul (?)
 - Who admires John? Paul and George admire John
- Verb combines with object first
- First entity is *object* $\rightarrow$ another function gives truth value for *subject*

Ditransitive verbs

- Examples: gives
- Type: $\langle e, \langle e, \langle e, t \rangle \rangle \rangle$

Lambda operator

Semantic type of lambda expressions

- If u is a variable of type σ
- and α is an expression of type τ
- then $[\lambda u. \alpha]$ is an expression of type $\langle \sigma, \tau \rangle$

Then, $[[[\lambda u. \alpha]]]^{M,g}$ denotes a function from things of type σ to things of type τ

- Informally, this denotes the function that takes an object o of type σ as input and as output gives whatever the denotation of α is, under the assumption that the variable u stands for o
- Here, variable u is bound by lambda operator

Terminology

- Expression: a well formed string in L_α of any type
- Formula: any expression of type t
- Examples:
 - $\lambda x. \text{loves}(r, x)$
 - Type: $\langle e, t \rangle$
 - This denotes the function that takes an entity d where $d \in D$ and maps it onto truth value T iff Ringo loves d = the characteristic function of the set of entities that Ringo loves
 - $\lambda x. \text{loves}(x, r)$
 - Type: $\langle e, t \rangle$
 - Denotes function that takes entity d where $d \in D$ and maps it onto the truth value T iff d loves Ringo = the characteristic function of the set of entities that love Ringo
 - $\lambda x. \neg \text{loves}(x, r)$
 - Type: $\langle e, t \rangle$
 - Denotes function that takes entity $d \in D$ and maps it onto truth value T iff d does not love Ringo = the characteristic function of the set of entities that don't love Ringo
 - $\lambda x. \text{guitarist}(x) \wedge \text{loves}(x, r)$
 - Type: $\langle e, t \rangle$
 - Denotes function that takes entity $d \in D$ and maps it onto truth value T iff d is a guitarist and d loves Ringo
 - = the characteristic function of the set of entities that are guitarists and love Ringo
 - $\lambda x. \text{spouseOf}(x)$
 - Type: $\langle e, e \rangle$
 - Denotes (partial) function that takes entity $d \in D$ and maps it onto d 's spouse
 - $\lambda x. \text{fatherOf}(\text{spouseOf}(x))$
 - Type: $\langle e, e \rangle$

- Denotes (partial) function that takes entity $d \in D$ and maps it onto the father of d 's (their) spouse, i.e. their father in law
- $\lambda y[\lambda x.loves(x, y)]$
 - Type: $\langle e, \langle e, t \rangle \rangle$
 - Equivalent to $\lambda y \lambda x.loves(x, y)$
 - Denotes the function that takes an entity $d_1 \in D$ and maps it to function that takes entity $d_2 \in D$ and maps it onto T iff d_2 loves d_1
 - = the function that takes entity $d_1 \in D$ and maps it onto (the characteristic function of) the set of entities that love d_1
 - Right to left curried denotation of the transitive verb loves
- $\lambda x[\lambda y.loves(x, y)]$
 - Type: $\langle e, \langle e, t \rangle \rangle$
 - The function that takes two arguments d_1 and d_2 , and yields the truth value 1 iff d_1 loves d_2
 - Note the difference with above - the order of the bound variables such that x is bound to outer lambda and y is bound to inner lambda here

Recall that in L_λ we can also have variables over expressions of other types besides entities/individuals

- Let P be a predicate variable, i.e., a variable over expressions of type $\langle e, t \rangle$
- $\lambda P.P(j)$
 - Type: $\langle \langle e, t \rangle, t \rangle$
 - Denotes the function takes the characteristic function of a set and maps it onto the truth value T iff John is a member of that set
 - The set of all sets that have John as a member
 - Informally, we can think of this as the set of all of John's properties
- $\lambda P.\forall x[guitarist(x) \rightarrow P(x)]$
 - Type: $\langle \langle e, t \rangle, t \rangle$
 - Denotes the function that takes the characteristic function of a set and maps it onto T iff every guitarist is a member of that set
 - Informally, the set of properties that every guitarist has

Lambda calculus

Lambda-expressions

Recall that in L_λ , we can also have variables over expressions of other types besides entities/individuals

Let P be a predicate variable, i.e., variable over expressions of type $\langle e, t \rangle$

Then $\lambda P.P(j)$ is of type $\langle \langle e, t \rangle, t \rangle$

- Denotes function that takes the characteristic function of a set and maps it onto the truth value T iff John is a member of that set
- = the set of all sets that have John as a member
- The set of all of John's properties

$\lambda P.\forall x[guitarist(x) \rightarrow P(x)]$ is of type $\langle \langle e, t \rangle, t \rangle$

- Denotes function that takes the characteristic function of a set and maps it onto T iff every guitarist is a member of that set
- = The set of properties that every guitarist has

Function application

- Syntax rule: For any types σ and τ , if α is an expression of type $\langle \sigma, \tau \rangle$ and β is an expression of type σ then $\alpha(\beta)$ is an expression of type τ
- Semantic rule: If α is an expression of type $\langle \sigma, \tau \rangle$, and β is an expression of type σ , then $[[\alpha(\beta)]]^{M,g} = [[\alpha]]^{M,g}([[\beta]]^{M,g})$
 - Then denotation of function alpha(beta) will be whatever denotation of alpha is, applied to beta

Lambda abstraction

- Syntax: If α is an expression of type τ and u is a variable of type σ then $[\lambda u.\alpha]$ is an expression of type $\langle \sigma, \tau \rangle$
 - We call σ and τ the input type and output type, respectively
- Semantic: If α is an expression of type τ and u is a variable of type σ , then for any assignment g , $[[\lambda u.\alpha]]^{M,g}$ is that function f from D_σ into D_τ such that for all objects o in D_σ , $f(o) = [[\alpha]]^{M,g[u \rightarrow o]}$

Example

Consider $\text{guitarist}(y)$

- Type: t
- Then α is of type t
- Then by abstraction, $\lambda y.\text{guitarist}(y)$
- Expression becomes of type $\langle e, t \rangle$
- Then consider denotation: $f = [[\lambda y.\text{guitarist}(y)]]^{M,g}$
- Which is the same as: $f(o) = [[\text{guitarist}(y)]]^{M,g[y \mapsto o]}$
 - Variable is replaced with that individual
 - Where $g = [y \mapsto \text{John}]$
 - Bound variables: get interpretation not from interpretation function g , but whatever argument is given to replace y with o , i.e., o becomes input to function f
 - Point of lambda abstraction is to bind variable
 - Put in some argument of this type
 - Replace all instances of that variable with object of type

See section 5.8 for full list of syntactic and semantic rules of L_λ

Lambda conversion/beta reduction

Example

$[\lambda x.\text{guitarist}(x) \wedge \text{woman}(x)](j)$

- Type: t
- In brackets: type $\langle e, t \rangle$
- (j) is type e
- Output (can then be reduced to): type t

is equivalent to

$\text{guitarist}(j) \wedge \text{woman}(j)$

- Type: t

Beta reduction allows us to make this equivalence

Definition

Using beta reduction, an expression of the form:

$$[\lambda x. \dots x \dots](\alpha)$$

can be simplified to

$\dots \alpha \dots$

Essentially - take value description (part to the right of the $.$) and replace all free occurrences of the λ -bound variable with the argument α

More examples

$$[\lambda x. loves(x, x)](j) = loves(j, j)$$

$$[\lambda x. loves(x, spouseOf(x))](j) = loves(j, spouseOf(j))$$

$$[\lambda y \lambda x. loves(x, y)](j)(p) = loves(p, j)$$

$$[\lambda P. P(j)](guitarist) = guitarist(j)$$

- Type in brackets: $\langle \langle e, t \rangle, t \rangle$
- guitarist type: $\langle e, t \rangle$

$$[\lambda P. P(j)](\lambda x. guitarist(x))$$

$$= [\lambda x. guitarist(x)](j)$$

$$= guitarist(j)$$

Thus, $(guitarist)$ and $\lambda x. guitarist(x)$ are equivalent

Restrictions on lambda conversion

1. Can only substitute α for those occurrences of the variable that are bound by λ
 - E.g., $[\lambda x. guitarist(x) \wedge \exists x[woman(x)]](j)$
 - $= guitarist(j) \wedge \exists x[woman(x)]$
 - Essentially, 2 'x' variables represent different things - 1 bound to λ and 1 bound to the existential
 - Solution: first rename variable
2. If α is a variable or contains a variable, it should not get bound "accidentally" as a result of the beta reduction
 - E.g., $[\lambda x. guitarist(x) \wedge \exists y[admires(x, y)]](y)$
 - $= guitarist(y) \wedge \exists y[admires(y, y)]$
 - $[\lambda x. guitarist(x) \wedge \exists z[admires(x, z)]](y)$
 - y then becomes bound after beta reduction
 - Solution: first rename variable

Function application

Frege's conjecture: the meaning of a complex expression is determined by the meanings of its parts and the way they are put together

Null hypothesis: there is only one way for the meanings of two subexpressions to combine to give the meaning of a complex expression: application of a function to an argument

Interpretation

Indirect interpretation steps

1. Translate expressions of English into expressions of L_λ
 - i. Kai $\rightsquigarrow$ k (type e)
 - ii. laughed $\rightsquigarrow \lambda x.\text{laughed}(x)$ (type $\langle e, t \rangle$)
2. The combination 'Kai laughed' will then be translated as the result of applying the translation of the verb to the translation of the subject

Composition rule 1: Function application

Let γ be a syntax tree whose only two subtrees are α and β (in any order) where

- $\alpha \rightsquigarrow \alpha'$ where α' has type $\langle \sigma, \tau \rangle$
- $\beta \rightsquigarrow \beta'$ where β' has type σ

Then $\gamma \rightsquigarrow \alpha'(\beta')$

Interpreting syntax

This fragment of syntax allow certain nodes to have only one daughter so:

Composition rule 2: Non-branching nodes (NN)

If β is a (sub-)tree whose only daughter is α , where $\alpha \rightsquigarrow \alpha'$, then $\beta \rightsquigarrow \alpha'$

Quantifiers

Everybody laughed $\rightsquigarrow \forall x.\text{laughed}(x)$

- By removing the contribution of laughed above, we are left with: $\forall x._ (x)$
- Reasoning backwards, the contribution of 'everybody' is then:
 - everybody $\rightsquigarrow \lambda P.\forall x.P(x)$ of type $\langle \langle e, t \rangle, t \rangle$

- Generalized quantifiers, i.e., property of properties
- Derivation for 'everybody laughed':

$$\begin{aligned} S &\rightsquigarrow [\lambda P \forall x [P(x)]] (\lambda y. \text{laughed}(y)) \\ &= \forall x [(\lambda y. \text{laughed}(y))(x)] \\ &= \forall x [\text{laughed}(x)] \end{aligned}$$

'a'

- Cannot treat 'a' as identity function
- Won't treat for sentences like: 'a singer laughed'
- Solution: equivalent to 'some'

Intersecting adjectives

Where two properties are intersecting

Example

1. Frida is Swedish
2. Frida is a singer
3. Frida is a Swedish singer

Intuitive truth conditions: $\text{swedish}(f) \wedge \text{singer}(f)$

- But both are type $\langle e, t \rangle$
- Type clash

Solution 1: A modifier type for adjectives

Swedish $\rightsquigarrow \lambda P \lambda x. \text{swedish}(x) \wedge P(x)$

- Type $\langle \langle e, t \rangle, \langle e, t \rangle \rangle$
- Advantages: technically works
- Disadvantages: but breaks other sentences that were fine, e.g., 'Frida is Swedish'

Solution 2: Ambiguity

Swedish₁ $\rightsquigarrow \lambda x. \text{swedish}(x)$

- $\langle e, t \rangle$ (like Frida is Swedish)

Swedish₂ $\rightsquigarrow \lambda P \lambda x. \text{swedish}(x) \wedge P(x)$

- Like Frida is a Swedish singer, type in solution 1

In general: want to avoid this kind of systematic ambiguity, especially for every lexical item for some class

Solution 3: A type-shifting approach

- Take one translation as 'basic' and derive the other one from it using a regular rule: type shift, or silent operator
- Type shifters are typically assumed to be invisible to syntax and are triggered by a type mismatch - not a piece of object language (nothing in syntax that's there)
- Silent operators may have a reflection in morpho-syntax, e.g., intransitive to transitive (would be a node in the syntax)

Type shifting rule: predicate-to-modifier shift (mod)

If $\alpha \rightsquigarrow \alpha'$, where α' is of type $\langle e, t \rangle$, then $\alpha \rightsquigarrow \lambda P \lambda x. \alpha'(x) \wedge P(x)$

Solution 4: a new composition rule

Predicate modification (PM): if γ is a tree whose only two subtrees are α and β where $\alpha \rightsquigarrow \alpha'$ and $\beta \rightsquigarrow \beta'$, and α' and β' are both type $\langle e, t \rangle$, then:

$$\gamma \rightsquigarrow \lambda u. \alpha'(u) \wedge \beta'(u)$$

- where u is a type e variable that does not occur free in α' or β'
- basically just combine

Relative clauses

Consider relative clauses as set intersection

- E.g., a reasonable doubt is a doubt which is reasonable $\rightarrow \lambda x. \text{reasonable}(x) \wedge \text{doubt}(x)$
- Also, woman who Bjorn loves $\rightarrow \lambda x. \text{woman}(x) \wedge \text{loves}(b, x)$
 - This is the end state

Assuming PM, simplest solution would be: $\lambda x. \text{loves}(b, x)$ of type $\langle e, t \rangle$

Relative clauses contain:

- A complementizer 'that'
- A relative pronoun 'which', 'who'
- One of these must be silent in English (doubly-filled comp filter)

Assume movement of relative pronoun from its base position to Spec, CP

- Relative pronoun co-indexed ($i \in \mathbb{N}$) with its trace
 - Trace carries same index as pronoun that moved

Deriving C' : see syntax tree

- Assume traces are translated as variables - tells us to replace traces with variables
- $t_3 \rightsquigarrow x_3$
- Recall: $[[x_3]]^{M,g} = g(x_3)$
- Then, the **pronouns and traces rule** is: if α is an indexed trace or pronoun, $\alpha_i \rightsquigarrow x_i$

Predicate abstraction: If γ is a syntax tree whose only two subtrees are α_i and β , α_i is a terminal node carrying index i , $\beta \rightsquigarrow \beta'$ where β' is an expression of any type, then $\gamma \rightsquigarrow \lambda x_i. \beta'$ where the index on α_i and x_i is the same

- α_i doesn't have a translation, it only carries its index, and triggers the predicate abstraction rule
- It is a syncategorematic expression, i.e., it has no translation, its effect is given by the rule
- Similar to intersective adjectives that are type $\langle e, t \rangle$ and modify noun/noun phrases

Quantifiers

For sentences with quantifiers in object position, e.g., "Bjorn loves every singer"

Quantifier raising

- We invoke a level of representation called **Logical Form**
- LF: level of syntactic interpretation that takes place 'after' or in parallel to phonological interpretation (PF) which is where words are pronounced
- At LF, words might appear in a different order from where they are pronounced, e.g., if movement has taken place/has been reconstructed at LF
- Movement that takes place at LF only is known as covert/silent movement
- Movement of quantifier leaves behind a type e trace
- An index is generated as the sister to the moved quantifier, triggering Predicate Abstraction rule

A type-shifting approach: Object raising

Rule: If an English expression α is translated into a logical expression α' of type $\langle e, \langle a, t \rangle \rangle$ for any type a , then α also has a translation of type $\langle \langle \langle e, t \rangle, t \rangle, \langle a, t \rangle \rangle$ of the following form:

$$\lambda Q_{\langle \langle e, t \rangle, t \rangle} \lambda x_a. Q(\lambda y. \alpha'(y))(x)$$

- Unless Q, y, or x occurs in α' ; in that case use different variables
- Applies to transitive verb, allows quantifier to just be argument to word (takes quantifier of type $\langle\langle e, t \rangle, t \rangle$ as its first argument

Another type-shifting approach: Subject raising

Rule: If an English expression α is translated into a logical expression α' of type $\langle a, \langle e, t \rangle \rangle$ for any type a, then α also has a translation of type $\langle a, \langle \langle \langle e, t \rangle, t \rangle, t \rangle \rangle$ of the following form:

$$\lambda y_a \lambda Q_{\langle \langle e, t \rangle, t \rangle} . Q(\lambda x . \alpha'(y)(x))$$

- Needed for inverse scope reading in the case of quantifier scope ambiguity
- Apply raise-s first then raise-o

Pronouns

Pronouns as variables, like traces

Rule: If α is an indexed trace or pronoun, $\alpha_i \rightsquigarrow x_i$

As variables, they receive their meaning from whatever they point to in a given assignment function g, e.g., $g = [x_1 \rightarrow \text{Emily}, x_2 \rightarrow \text{Sadie}]$

Use of pronouns

Deictic: reference established by properties of the discourse situation, e.g., speech act participants, pointing

- Examples: [Context: Pointing to T] He is late.; You won the grand prize!

Anaphoric: reference established by introducing or making an individual salient in the discourse, then referring back to them using a pronoun

- Examples: Matthias walked into the classroom. He sat down.
- In this case, Matthias is the antecedent of he

Some uses of pronouns are not referential at all (bound pronouns)

- Examples: No woman blamed herself; Neither man thought he was at fault; Every boy loves his mother
- In the first example, the intuitive translation is: $\neg \exists x [\text{woman}(x) \wedge \text{blamed}(x, x)]$
 - Then, x is clearly bound by $\exists$, i.e., x does not get its interpretation from the assignment function g

Analogy to 'such that'

- 'Book such that Mary read it' -> same truth conditions as relative clause, 'book that Mary read'

Free and bound variables

Pronouns interpreted as bound will occur in the scope of an instance of predicate/lambda abstraction

Otherwise, we will say that the pronoun (variable) is free

Lift

Type-shift rule: If an English expression α is translated as an individual constant α' of type e, then α also has a translation of type $\langle \langle e, t \rangle, t \rangle$ of the form:

$$\lambda P_{\langle e, t \rangle}. P(\alpha')$$

- Just changes type, doesn't really do anything
- Not triggered by type mismatch like other type shifting rules, but motivated by our want to move this constant as if it's a quantifier, thus needs the quantifier type

Presuppositions

Implementing presuppositions

Dealing with presupposition failure

- Add additional truth failure $\#_t$ (Kleene) alongside T, F
- The truth value of a sentence containing a presupposition failure will be $\#_t$
- In the case of false presupposition (e.g., king of france is bald), the presupposition failure comes from a failure of the definite description to denote anything
- We can also add an undefined individual $\#_e$ to our domain

Definite determiner

We will incorporate the iota operator ι into the translation of the definite determiner 'the' -> 'the unique' - whole thing is type e

- E.g., the orange cat $\rightsquigarrow \iota x. \text{orange}(x) \wedge \text{cat}(x)$

Syntax rule: If ϕ is an expression of type t and u is a variable of type τ , then $\iota u. \phi$ is an expression of type τ

Semantic rule: If ϕ is an expression of type t and u is a variable of type τ , then

$$[[\iota u.\phi]]^{M,g} =$$

- d if $[[\phi]]^{M,g[u \mapsto d]} = T$ but
 - for all $d' \in D_\iota$ distinct from d , $[[\phi]]^{M,g[u \mapsto d']} = F$
- $\#_\tau$ otherwise

ι encodes existence and uniqueness, and returns $\#_\tau$ if those conditions are not met

For the definite determiner, we encode ι in the translation relative to individuals of type e

Then, 'the' $\rightsquigarrow \lambda P.\iota x.P(x)$

- Of type $\langle\langle e, t \rangle, e\rangle$

When a DP denoting $\#_e$ occurs within a sentence, the truth value of that sentence will be $\#_t$

- Thus the following sentences (which have an undefined value relative to the actual world) denote $\#_t$
 - The king of France is bald
 - The classroom in OC is locked

Generalized definedness conditions

We would like to generalize the representation of presuppositions

Not all presuppositions have to do with existence and uniqueness, and not all presuppositions are based on the definedness conditions of individuals

The partial operator ∂

Syntax rule: If ϕ is an expression of type t , then $\partial(\phi)$ is an expression of type t

Semantic rule: If ϕ is an expression of type t , then

$$[[\partial(\phi)]]^{M,g} =$$

- $\top$ if $[[\phi]]^{M,g} = T$
- $\#$ otherwise

[Context: Frida, Benny, and Björn are running in the election for president of their knitting club]

a. $\#_{\text{Both}}$ candidates are qualified

b. #Neither candidate is qualified

Informally: both and neither presuppose that the cardinality of the set characterized by their first argument is exactly 2

- We can model this using ∂ as follows:
- $\partial(|P| = 2)$ ($\approx$ "presupposing the cardinality of P is 2")

'neither' $\rightsquigarrow \lambda P \lambda P' [\partial(|P| = 2) \wedge \neg \exists x. [P(x) \wedge P'(x)]]$

'some' $\rightsquigarrow \lambda P \lambda P' [\partial(|P| = 2) \wedge \exists x. [P(x) \wedge P'(x)]]$

'every' $\rightsquigarrow \lambda P \lambda P' [\partial(|P| = 2) \wedge \forall x. [P(x) \rightarrow P'(x)]]$

Alternate notation: moves cardinality argument to expression (no partial)

'neither' $\rightsquigarrow \lambda P : |P| = 2. \lambda P' [\neg \exists x. [P(x) \wedge P'(x)]]$

- But critique is that by reduction/application, the presupposition goes away

Event semantics

Davidson (1967)!

- Sentences are fundamentally about events
- Verbs are predicates of events (functions from events to truth values)

Adding events to our system:

- Variables: $e, e', \dots$
- Type: v

Example: $\text{sing} \rightsquigarrow \lambda e. \text{sing}(e)$

- Type $\langle v, t \rangle$
- We read $\text{sing}(e)$ as: "e is an event of singing"

The correct representation of verb meaning, and specifically of the arguments of verbs is still up for debate in the literature:

- $\lambda x \lambda y \lambda e. \text{butter}(y, x, e)$
 - Add event argument e to existing representation
- $\lambda x \lambda e. \text{butter}(x, e)$

- Only object and event as arguments of the verb
- In favor of: plausible representation; most common in literature
- $\lambda e.\text{butter}(e)$
 - Only event as argument of the verb

Object-dependent VP interpretations

- Throw
 - Throw a baseball
 - Throw support behind a candidate
 - Throw a boxing match
 - Throw a party
 - Throw a fit
- Take
 - Take a book from the shelf
 - Take a bus to Burnaby
 - Take a nap
 - Take an aspirin
 - Take notes
- Kill
 - Kill a cockroach
 - Kill a conversation
 - Kill an evening watching TV
 - Kill a bottle

The **internal argument** (object) triggers a particular interpretation of the VP

- Few parallel examples involving external arguments

Nominalizations

Example:

- We enjoyed his reading of Pride and Prejudice
- We enjoyed his reading Pride and Prejudice
- We enjoyed him reading Pride and Prejudice

Consider a situation in which individual referred to by his/him is instead one of several people in charge of organizing readings of Pride and Prejudice

- Then, the first can be used to describe the situation; the latter two cannot

- This shows that his in the first is not semantically an agent, but rather a true possessor -> level of representation of the VP without the agent

These are then arguments for the above verb representation, where x is the object

- butter $\rightsquigarrow \lambda x \lambda e. \text{butter}(x, e)$
- e is an event of buttering x

θ -solution

Main idea: assume that nominal arguments can be introduced by (silent) functions that also introduce their thematic roles

$$[\text{agent}] \rightsquigarrow \lambda x \lambda e. \text{agent}(e) = x$$

Existential closure

The type we have derived for a sentence is now $\langle v, t \rangle$

Type-shift rule - Existential closure: If $\alpha \rightsquigarrow \alpha'$, where α' is of type $\langle v, t \rangle$, then:

$$\alpha \rightsquigarrow \exists e. \alpha'(e)$$

Extending our system - event modifiers

We can now add modifiers (adverbs?) like slowly to our system:

$$\text{slowly} \rightsquigarrow \lambda e. \text{slow}(e)$$

$$\text{in the kitchen} \rightsquigarrow \lambda e. \text{in}(e, \iota y. \text{kitchen}(y))$$

$$\text{in} \rightsquigarrow \lambda x \lambda e. \text{in}(x, e)$$

Modifiers can be added as adjunct to VP or S -> still get same truth conditions since we're just conjoining them by predicate modification (all it needs is sister of type vt)

Little v

Arguments for severing the external argument from the verb are the same ones used to argue for v in the syntax

$$v_{\text{active}} \rightsquigarrow \lambda \mathcal{P}_{\langle v, t \rangle} \lambda x. \lambda e. \mathcal{P}(e) \wedge \text{agent}(e) = x$$

Other flavors of v

Frida owns the book

- Frida isn't an agent, but a holder of a state
- $v_{\text{state}} \rightsquigarrow \lambda \mathcal{P} \lambda x \lambda e. \mathcal{P}(e) \wedge \text{holder}(e) = x$

The toast was buttered by Benny

- $v_{\text{passive}} \rightsquigarrow$
- by $\rightsquigarrow$

Tense and aspect

Aktionsart (lexical aspect)

Categorization of event types named by a VP

States: if a stative predicate is true at time interval I, then it is true at all subintervals of I -> non-dynamic; do not entail a change of any kind in an argument of the verb

- Example: Benny knows French.

Activities: if an activity predicate is true at a time I, then it is true at all subintervals of I up to a certain size -> dynamic; entail some change in an argument of the verb

- Example: Frida danced.

Accomplishment/achievement: if true at I, then false at all subintervals of I

- Accomplishments: protracted in time
 - Example: Benny drew a circle.
- Achievements: conceived of as instantaneous
 - Example: Frida noticed the painting.

Properties of a verb's arguments can affect Aktionsart:

- Agentha ate.
- Agnetha ate pancakes.
- Agnetha ate five pancakes. -> names quantity of object - event ends after five pancakes

Defining tense

- Comrie (1985): The grammaticalization of location in time
 - Q1: location of what?
 - Q2: location w.r.t. what?
- First pass at answering Q1/Q2: Tenses locate events relative to utterance time
 - Utterance time (UT): time at which an utterance is made
 - Eventuality time (ET): time at which an eventuality (event or state) is located (also known as situation time - ST)
 - Kim danced. $ET < UT$
 - Kim has danced. $ET < UT$
 - Where $<$ is a temporal precedence relation
 - Reads, ET is prior to/before UT
 - Problem: 1) Yesterday Kim danced. 2) Yesterday Kim has danced
- Adding Reference time (RT) also known as Topic time (TT)
 - Determination of time point is given by context of speech (Reichenbach)
 - RT is the time the (chunk of) discourse is "about" (Klein)
 - Time adverbials like 'now', 'yesterday', 'November 5, 1605' identify RT
 - E.g., 1) I saw you in the park by the water fountain yesterday. 2) I was walking my dog.
- Revised analysis: tense locates RT w.r.t. UT; aspect locates ET w.r.t. RT
 - Kim danced. $ET = RT < UT$
 - Eventuality time is the reference time which is before the utterance time
 - Kim has danced. $ET < RT = UT$
 - Yesterday Kim has danced - temporal adverbs locate RT so RT is present in this
- Current hypotheses:
 - Present tense: $RT = UT$
 - Past tense: $RT < UT$
 - Assuming (under Reichenbach) that deictic temporal adverbs constrain the location of RT w.r.t. UT, then the use of such adverbs should constrain RT in at least the following ways:
 - Yesterday: $RT < UT$
 - Tomorrow: $UT < RT$
 - Right now: $RT \bigcirc UT$ where $\bigcirc$ = temporal overlap
 - Will not allow for e.g., 'Tomorrow Kim danced' and 'Right now Kim danced'

Tense vs aspect

Other verbal forms, e.g., progressive and perfect

- Progressive (BE + present participle)

- Kim was dancing.
- Kim is dancing.
- Kim will be dancing.
- Perfect (HAVE + past participle)
 - Kim had danced.
 - Kim has danced.
 - Kim will have danced.

Progressive and perfect are **aspects**

- Relations between RT and ET (grammatical aspect)
- Combine with tenses to give more information about the temporal profile of events
- Progressive: $RT \subseteq ET$
- Perfect: $ET < RT$
- Perfective: $ET \subset RT$

Developing a framework

	type	variables
individuals	e	x, x', y, y', ...
events	v	e, e', ...
times	i	t, t', ...
truth values	t	p, q, ...

Aspect: quantifier over events

- Relation between ET and RT
- Function from predicate of events to a predicate of times
 - Type: $\langle \langle v, t \rangle, \langle i, t \rangle \rangle$
 - Existentially quantifies over event variable
- $PROG \rightsquigarrow \lambda \mathcal{P} \lambda t. \exists e [\mathcal{P}(e) \wedge \tau(e) \supseteq t]$
 - Approximately equals IPFV/imperfective
 - t represents reference time, i.e., $t = RT$
 - $\tau(e) = ET$
 - So same as $RT \subseteq ET$
- $PFV \rightsquigarrow \lambda \mathcal{P} \lambda t. \exists e [\mathcal{P}(e) \wedge \tau(e) \subset t]$
- $PERF \rightsquigarrow \lambda \mathcal{P} \lambda t. \exists e [\mathcal{P}(e) \wedge \tau(e) < t]$

- Where τ = "temporal trace function" and $\tau(e)$ = the runtime of e

Tense: quantifier over times

- Relation between RT and UT (now)
- Hypothesis: existential quantifier over times
- Type: $\langle \langle i, t \rangle, t \rangle$
- PAST $\rightsquigarrow \lambda Q_{\langle i, t \rangle}. \exists t [Q(t) \wedge t < UT]$
- PRESENT $\rightsquigarrow \lambda Q. \exists t [Q(t) \wedge t = UT]$
- FUTURE $\rightsquigarrow \lambda Q. \exists t [Q(t) \wedge t > UT]$
- yesterday $\rightsquigarrow \lambda t. t \subseteq \text{yesterday}$ which is of type $\langle i, t \rangle$

Pronominal analysis of tense

Some structural analogies between tenses and pronouns in English (Partee, 1973)

1. Deictic pronouns and tenses

- **I am** standing in the room
 - Deictic interpretation - time of utterance (present tense) and utterer (I)
- He shouldn't be in here
 - Deictic use of third-person pronoun; accompanied by pointing to person
- I didn't turn off the stove
 - $\exists$ -TCs (truth conditions) for PAST is either too weak or too strong (depending on how we interpret negation relative to existential)
 - Too strong: $\neg \exists t$
 - Too weak (trivial): $\exists t. \neg$
 - Intuitive meaning of this sentence isn't captured by truth conditions we've developed and negation

2. Anaphoric pronouns and tenses

- Sam took **the car** yesterday, and Sheila took **it** today.
- Sheila had a party **last Friday**, and Sam **got drunk**.
- When Susan walked in, Peter **left**.
- He who stole my cow, **he** will suffer the penalties

3. Pronouns and tenses as bound variables

- If one of the arrows hits the target, it's mine
- Whenever Susan comes in, John immediately leaves.
- Whenever Susan came in, John immediately left.
- No one could tell what he was being tested for.
- John never answers when I call his home.

Personal pronouns: individual pronoun (he, she, I, they - variable) + person, gender, number features

Tenses: temporal pronoun + tense features

Pronominal theory of tense: tenses directly denote temporal pronouns and receive their reference from the assignment function g

$PAST_i \rightsquigarrow g(i)$; defined only if $g(i) < UT$